DEP512E – HW1 Part A
Newmark–β Analysis of a SDOF System

Mesut ADALI - 801262107

19.10.2026

1 Method

The equation of motion is

$$m\ddot{x} + c\dot{x} + kx = p(t), \qquad p(t) = -m\ddot{x}_g(t) \text{ for ground motion.}$$

The Newmark–β method (incremental form) is used with $\beta = 1/4$ and $\gamma = 1/2$. At each time step:

$$\hat{k} = k + \frac{\gamma}{\beta\Delta t}c + \frac{m}{\beta\Delta t^2}, \qquad \Delta\hat{p}_i = \Delta p_i + A\,\dot{x}_i + B\,\ddot{x}_i, \qquad \Delta x_i = \frac{\Delta\hat{p}_i}{\hat{k}}$$

$$A = \frac{m}{\beta\Delta t} + \frac{\gamma}{\beta}c, \qquad B = \frac{m}{2\beta} + \Delta t\left(\frac{\gamma}{2\beta} - 1\right)c$$

$\Delta\dot{x}_i$ and $\Delta\ddot{x}_i$ follow from Δx_i, and $x_{i+1} = x_i + \Delta x_i$ (similarly for $\dot{x}$ and $\ddot{x}$).

2 Code

- `newmark(m,c,k,p,dt,beta,gamma,x0,v0)`: applies the steps above and returns the displacement, velocity and acceleration vectors.
- Main part: defines the system and the loads, calls `newmark` and plots the results. The spring force is $f_S = kx$ and the damper force is $f_D = c\dot{x}$.
- System: $m = 1$ kg, $\omega_n = 2\pi$ rad/s, $\xi = 0.05$, so $k = m\omega_n^2 = 39.48$ N/m and $c = 2\xi m\omega_n = 0.628$ N s/m.
- Problem 1: $p(t) = 2\sin(\pi t/2)$, $\Delta t = 0.01$ s, 20 s.
- Problem 2: ground acceleration from `ground_acc.txt`, $\Delta t = 0.005$ s (7860 points, 39.3 s), $p = -m\ddot{x}_g$.

3 Results – Problem 1

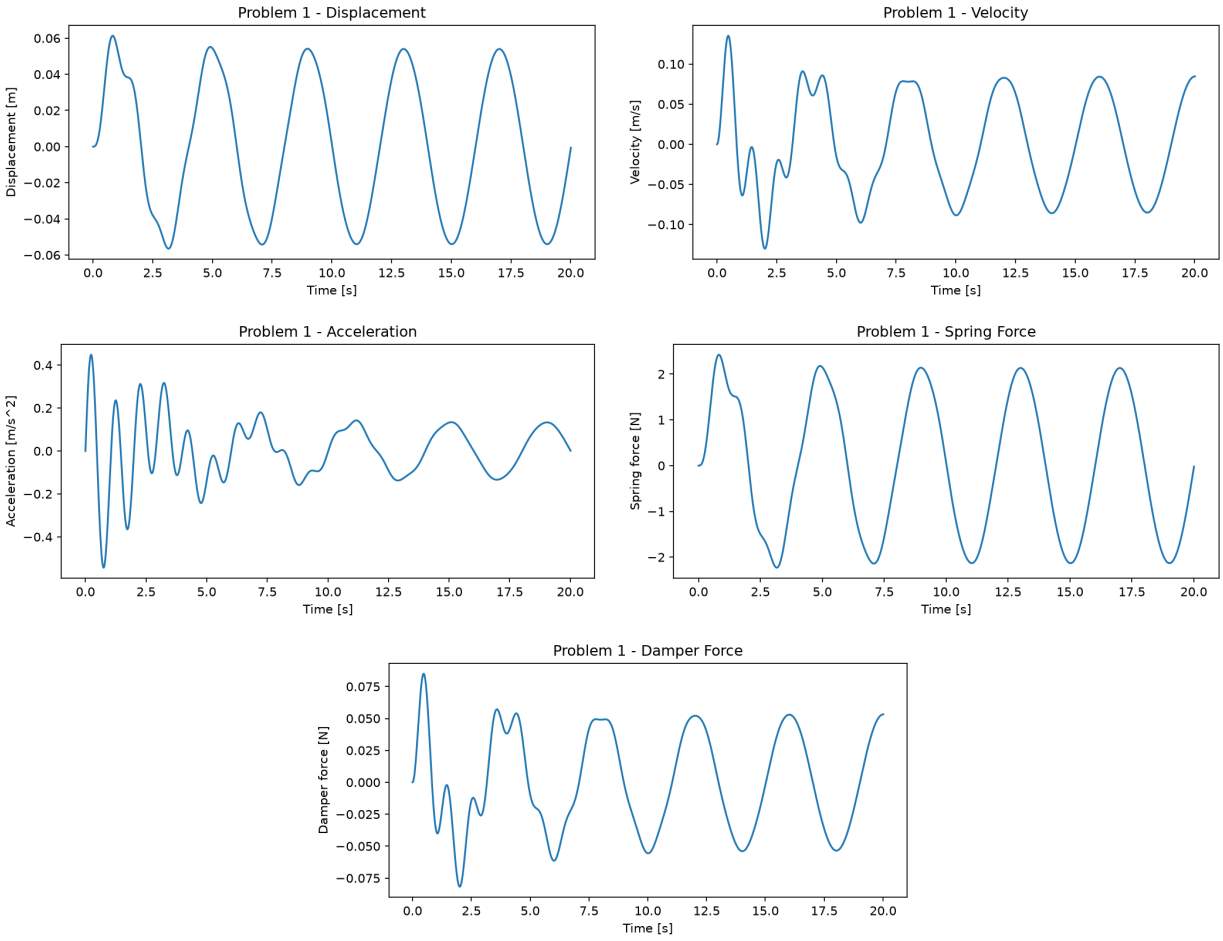


Figure 1: Problem 1: time histories.

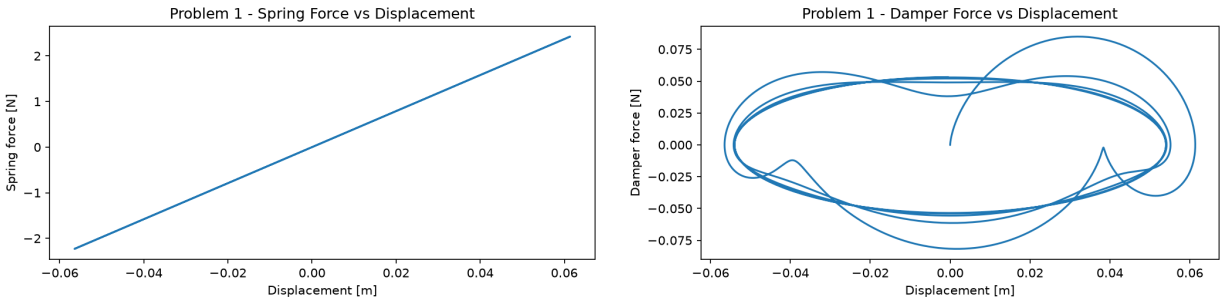


Figure 2: Problem 1: displacement vs spring force and damper force.

4 Results – Problem 2

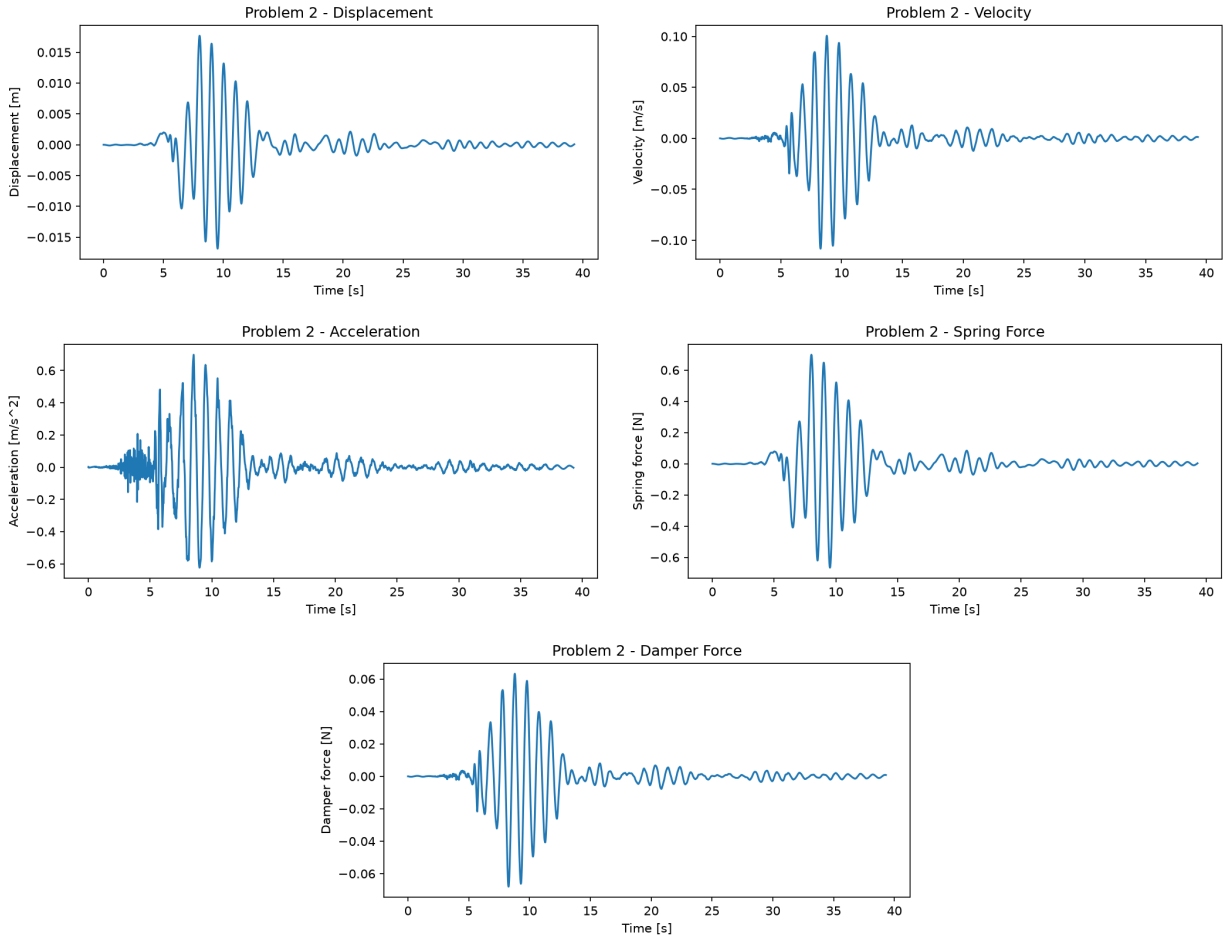


Figure 3: Problem 2: time histories.

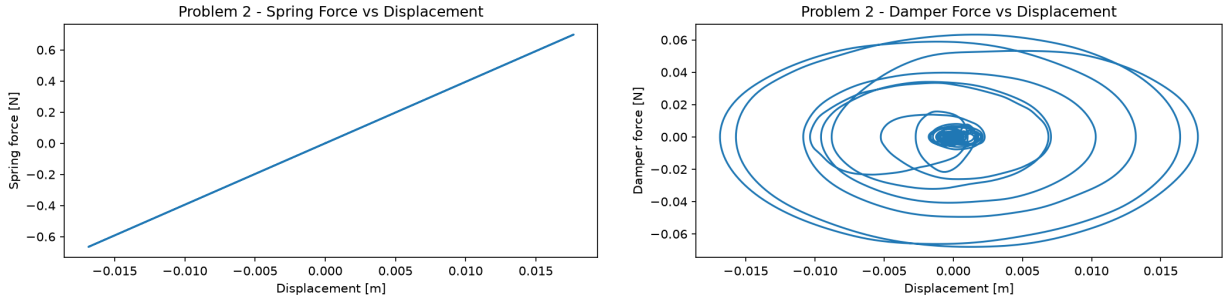


Figure 4: Problem 2: displacement vs spring force and damper force.

A Python Code

```
1  import os
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  os.chdir(os.path.dirname(os.path.abspath(__file__)))
7
```

```
# Newmark parameters
beta = 1 / 4
gamma = 1 / 2

# wn = 2*pi rad/s, xi = 5 %
m = 1.0
wn = 2 * np.pi
xi = 0.05
k = m * wn ** 2
c = 2 * xi * m * wn

def newmark(m, c, k, p, dt, beta, gamma, x0, v0):
    n = len(p)
    x = np.zeros(n)
    v = np.zeros(n)
    a = np.zeros(n)

    # initial conditions
    x[0] = x0
    v[0] = v0
    a[0] = (p[0] - c * v0 - k * x0) / m

    # effective stiffness and constants
    k_hat = k + gamma / (beta * dt) * c + m / (beta * dt**2)
    A = m / (beta * dt) + gamma / beta * c
    B = m / (2 * beta) + dt * (gamma / (2 * beta) - 1) * c

    for i in range(n - 1):
        dp_hat = (p[i + 1] - p[i]) + A * v[i] + B * a[i]
        dx = dp_hat / k_hat
        dv = gamma / (beta * dt) * dx - gamma / beta * v[i] + dt * (1 - gamma / (2 * beta)) * a[i]
        da = dx / (beta * dt**2) - v[i] / (beta * dt) - a[i] / (2 * beta)

        x[i + 1] = x[i] + dx
        v[i + 1] = v[i] + dv
        a[i + 1] = a[i] + da

    return x, v, a
# Problem 1: p(t) = P*sin(wbar*t)
P = 2.0
wbar = np.pi / 2
dt = 0.01
t = np.arange(0, 20 + dt, dt)
p = P * np.sin(wbar * t)

x, v, a = newmark(m, c, k, p, dt, beta, gamma, 0, 0)
plt.figure()
plt.plot(t, x)
plt.title("Problem 1 - Displacement")
plt.xlabel("Time [s]")
plt.ylabel("Displacement [m]")

plt.figure()
plt.plot(t, v)
plt.title("Problem 1 - Velocity")
plt.xlabel("Time [s]")
plt.ylabel("Velocity [m/s]")

plt.figure()
plt.plot(t, a)
plt.title("Problem 1 - Acceleration")
plt.xlabel("Time [s]")
plt.ylabel("Acceleration [m/s^2]")

fs = k * x # spring force
```

```
fd = c * v # damper force

plt.figure()
plt.plot(t, k * x)
plt.title("Problem 1 - Spring Force")
plt.xlabel("Time [s]")
plt.ylabel("Spring force [N]")

plt.figure()
plt.plot(x, k * x)
plt.title("Problem 1 - Spring Force vs Displacement")
plt.xlabel("Displacement [m]")
plt.ylabel("Spring force [N]")


plt.figure()
plt.plot(t, c * v)
plt.title("Problem 1 - Damper Force")
plt.xlabel("Time [s]")
plt.ylabel("Damper force [N]")

plt.figure()
plt.plot(x, c * v)
plt.title("Problem 1 - Damper Force vs Displacement")
plt.xlabel("Displacement [m]")
plt.ylabel("Damper force [N]")


plt.show()

 #Problem 2: ground acceleration (m/s^2), dt = 0.005 s
dt = 0.005
ag = np.loadtxt("ground_acc.txt")
t = np.arange(len(ag)) * dt
p = -m * ag

x, v, a = newmark(m, c, k, p, dt, beta, gamma, 0, 0)
plt.figure()
plt.plot(t, x)
plt.title("Problem 2 - Displacement")
plt.xlabel("Time [s]")
plt.ylabel("Displacement [m]")

plt.figure()
plt.plot(t, v)
plt.title("Problem 2 - Velocity")
plt.xlabel("Time [s]")
plt.ylabel("Velocity [m/s]")

plt.figure()
plt.plot(t, a)
plt.title("Problem 2 - Acceleration")
plt.xlabel("Time [s]")
plt.ylabel("Acceleration [m/s^2]")

fs = k * x # spring force
fd = c * v # damper force

plt.figure()
plt.plot(t, k * x)
plt.title("Problem 2 - Spring Force")
plt.xlabel("Time [s]")
plt.ylabel("Spring force [N]")

plt.figure()
```

```
plt.plot(x, k * x)
plt.title("Problem 2 - Spring Force vs Displacement")
plt.xlabel("Displacement [m]")
plt.ylabel("Spring force [N]")


plt.figure()
plt.plot(t, c * v)
plt.title("Problem 2 - Damper Force")
plt.xlabel("Time [s]")
plt.ylabel("Damper force [N]")

plt.figure()
plt.plot(x, c * v)
plt.title("Problem 2 - Damper Force vs Displacement")
plt.xlabel("Displacement [m]")
plt.ylabel("Damper force [N]")

plt.show()
```